

TRABAJO PRÁCTICO INTEGRADOR

BASE DE DATOS I

**Dominicale Doré, María Luz
Egea Ruiz, Magaly
Ferrario, Inés**

23/06/2026

Introducción	5
Resumen Ejecutivo	5
Reglas de Negocio	5
Etapas 1 – Modelado y Constraints	7
Modelo Entidad–Relación (DER)	7
Modelo Relacional	8
Validación mediante inserciones	8
Etapas 2 – Carga Masiva	9
Mecanismo utilizado	9
Verificaciones de consistencia	10
Etapas 3 – Consultas Avanzadas	11
EXPLAIN y medición con/sin índice	14
Tabla de tiempos (Etapas 3)	16
Etapas 4 – Seguridad e Integridad	17
Usuario con privilegios mínimos	17
Vistas para ocultar información sensible	17
Pruebas de integridad	18
Procedimiento seguro anti-inyección	19
Etapas 5 – Concurrencia y Transacciones	20
Simulación de deadlock	20
Procedimiento con retry	20
Comparación de niveles de aislamiento	21
Conclusión General del TFI	23
6. Anexo IA	24
Estoy armando el DER y no sé si conviene usar el mail como PK o un id numérico. ¿Qué sería mejor a nivel diseño?	24
¿Cómo agrego un CHECK para asegurar que el stock nunca sea negativo?	26
¿Cómo puedo generar miles de registros sin escribirlos uno por uno? ¿Hay alguna técnica simple con INSERT...SELECT?	27
¿Cómo hago para generar fechas aleatorias dentro de los últimos 30 días?	30
¿Cómo verifico que no se generaron FK huérfanas después de la carga masiva?	32
No logro hacer un JOIN múltiple entre pedido, usuario y detalle_pedido para obtener el total del pedido	33
¿Me explicás el EXPLAIN? No entiendo qué significa que el tipo sea “ALL” o “ref”.	35
¿Cómo creo un usuario con privilegios mínimos para que solo pueda ver vistas y crear pedidos?	38
No sé cómo hacer un procedimiento almacenado que sea seguro y no permita SQL injection	40
Estoy intentando simular un deadlock pero no aparece el error 1213. No sé que estoy haciendo mal	42
¿MariaDB no reconoce la variable transaction_isolation?	45
¿Qué diferencia hay entre READ COMMITTED y REPEATABLE READ?	47

Introducción

El presente Trabajo Final Integrador (TFI) tiene como objetivo aplicar de manera progresiva los contenidos de Bases de Datos I, integrando diseño, implementación, carga masiva, consultas avanzadas, seguridad y concurrencia.

El sistema desarrollado, denominado Food Store, permite gestionar usuarios, productos y pedidos, simulando un entorno real de comercio.

El trabajo se desarrolló siguiendo las cinco etapas propuestas por la cátedra, utilizando MySQL Workbench como motor de base de datos y aplicando buenas prácticas de integridad, seguridad y rendimiento.

Resumen Ejecutivo

El presente Trabajo Final Integrador tiene como objetivo diseñar e implementar una base de datos relacional para el sistema Food Store utilizando MySQL. El proyecto contempla el modelado de la información, la generación de más de 10.000 registros para pruebas de rendimiento y el desarrollo de consultas avanzadas orientadas a la obtención de información útil para la gestión del sistema. Asimismo, se aplican mecanismos de seguridad e integridad de datos, junto con pruebas de concurrencia y transacciones, permitiendo evaluar el comportamiento de la base de datos en escenarios similares a los de un entorno real.

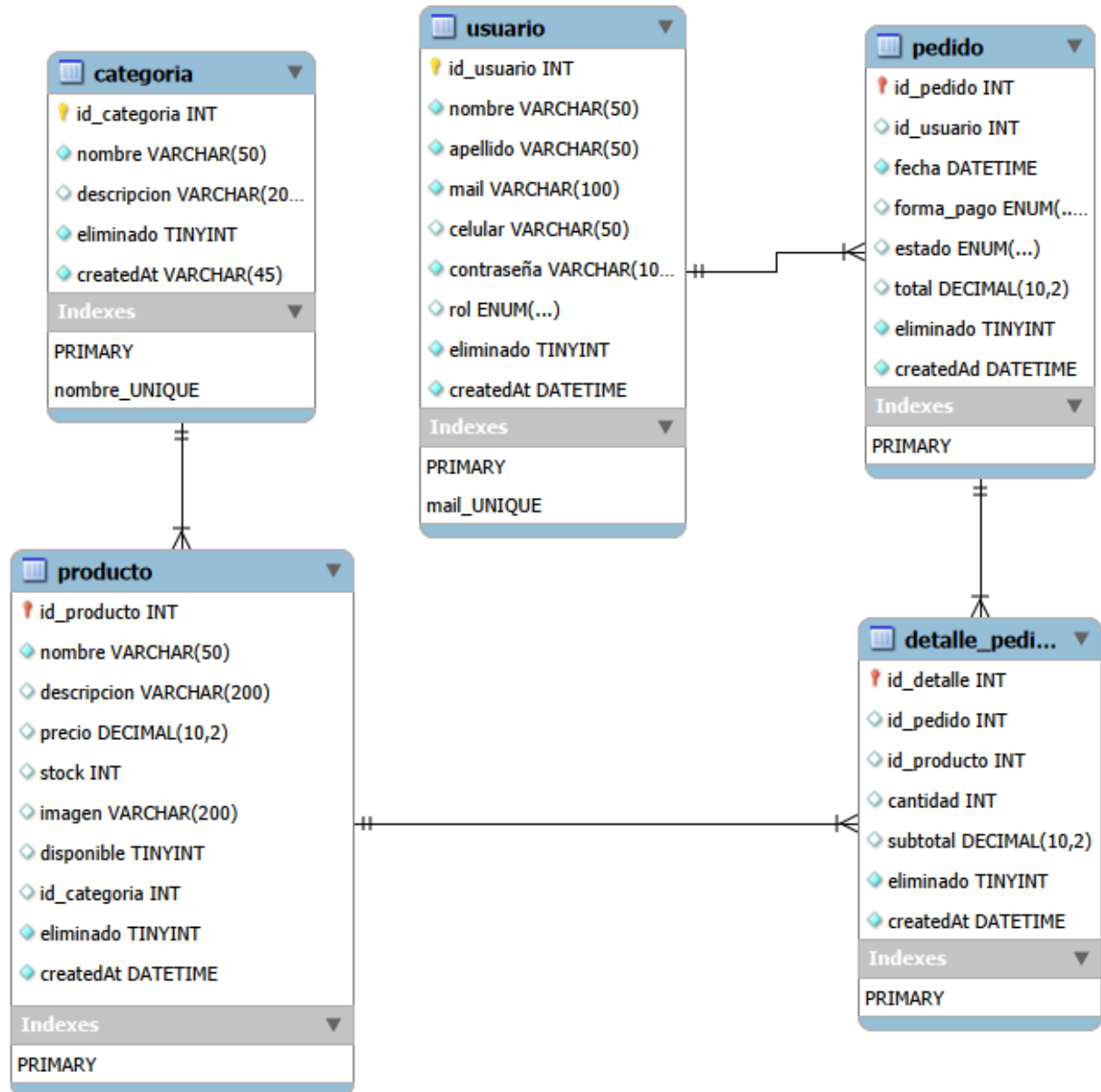
Reglas de Negocio

1. Cada usuario debe estar identificado de manera única dentro del sistema mediante un correo electrónico no repetido.
2. Cada categoría debe poseer un nombre único para evitar duplicidades en la clasificación de productos.
3. Todo producto debe pertenecer a una categoría válida registrada en el sistema.
4. El stock de los productos debe ser mayor o igual a cero en todo momento.
5. Un pedido debe estar asociado a un único usuario registrado.
6. Todo pedido debe contener al menos un detalle de pedido.
7. Cada detalle de pedido debe hacer referencia a un producto existente.
8. La cantidad solicitada de un producto no puede superar el stock disponible al momento de confirmar el pedido.
9. El importe total de un pedido se calcula como la suma de los subtotales de sus detalles.

10. Las eliminaciones de usuarios, categorías, productos y pedidos se realizan mediante baja lógica, preservando la información histórica almacenada en el sistema.

Etapa 1 – Modelado y Constraints

Modelo Entidad–Relación (DER)



Modelo Relacional

Se definieron las tablas con sus claves primarias, claves foráneas, restricciones UNIQUE, CHECK y restricciones de dominio (tipos, tamaños y obligatoriedad).

The screenshot shows a database management tool interface. On the left, the 'SCHEMAS' pane displays a tree view of the database structure, including 'libreria', 'phpmyadmin', 'test', and 'tpi_bd'. The 'tpi_bd' database is selected, showing its tables: 'categoria', 'detalle_pedido', 'pedido', 'producto', and 'usuario'. The 'Information' pane shows 'No object selected'. The main editor displays the SQL code for creating the 'detalle_pedido' table, including primary and foreign key constraints. The 'Output' pane shows the execution results of the SQL commands, including the creation of the database and tables, and the execution of the 'USE' statement.

```
54
55 -- Tabla de detalles del pedido
56 CREATE TABLE detalle_pedido (
57     id_detalle INT AUTO_INCREMENT PRIMARY KEY,
58     id_pedido INT NOT NULL,
59     id_producto INT NOT NULL,
60     cantidad INT NOT NULL CHECK (cantidad > 0),
61     subtotal DECIMAL(10,2) NOT NULL,
62     eliminado BOOLEAN NOT NULL DEFAULT 0,
63     createdAt DATETIME NOT NULL DEFAULT NOW(),
64     FOREIGN KEY (id_pedido) REFERENCES pedido(id_pedido),
65     FOREIGN KEY (id_producto) REFERENCES producto(id_producto)
66 );
```

#	Time	Action	Message	Duration / Fetch
1	08:57:47	CREATE DATABASE tpi_bd	1 row(s) affected	0.297 sec
2	08:57:50	USE tpi_bd	0 row(s) affected	0.000 sec
3	08:57:58	CREATE TABLE categoria (id_categoria INT AU...	0 row(s) affected	0.172 sec
4	08:58:04	CREATE TABLE usuario (id_usuario INT AUTO_I...	0 row(s) affected	0.125 sec
5	08:58:12	CREATE TABLE producto (id_producto INT AUT...	0 row(s) affected	0.141 sec
6	08:58:17	CREATE TABLE pedido (id_pedido INT AUTO_I...	0 row(s) affected	0.141 sec
7	08:58:23	CREATE TABLE detalle_pedido (id_detalle INT A...	0 row(s) affected	0.125 sec

Validación mediante inserciones

Se realizaron inserciones válidas e inválidas para comprobar:

- Violación de UNIQUE
- Violación de CHECK
- Violación de FOREIGN KEY

Ejecución de consultas de control para verificar la consistencia del conjunto de datos luego de la carga masiva.

TPI Bases de Datos

SQLAdditions

My Snippets

```

89 • SELECT COUNT(*) AS productos_con_categoria_invalida
90 FROM producto p
91 LEFT JOIN categoria c ON p.id_categoria = c.id_categoria
92 WHERE c.id_categoria IS NULL;
93
94 • SELECT COUNT(*) AS pedidos huierfanos

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Result Grid

productos_con_categoria_invalida

0

Result 1 x | Read Only | Context Help | Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 6	08:58:17	CREATE TABLE pedido (id_pedido INT AUTO...	0 row(s) affected	0.141 sec
✓ 7	08:58:23	CREATE TABLE detalle_pedido (id_detalle IN...	0 row(s) affected	0.125 sec
✓ 8	08:59:38	INSERT INTO usuario (nombre, apellido, mail, cel...	1 row(s) affected	0.094 sec
✗ 9	08:59:44	INSERT INTO usuario (nombre, apellido, mail, cel...	Error Code: 1062. Duplicate entry 'ana@mail.com' ...	0.063 sec
✗ 10	08:59:47	INSERT INTO producto (nombre, descripcion, pre...	Error Code: 4025. CONSTRAINT 'producto.precio...	0.015 sec
✗ 11	08:59:51	INSERT INTO producto (nombre, descripcion, pre...	Error Code: 1452. Cannot add or update a child ro...	0.062 sec
✗ 12	08:59:54	INSERT INTO pedido (id_usuario, fecha, forma_p...	Error Code: 1048. Column 'id_usuario' cannot be null	0.015 sec
✓ 13	09:00:18	SELECT COUNT(*) AS productos_con_categoria...	1 row(s) returned	0.063 sec / 0.000 sec

Etapas 2 – Carga Masiva

Se generó un conjunto de datos masivo utilizando SQL puro, con más de 10.000 registros distribuidos entre las tablas principales del sistema.

Mecanismo utilizado

- Tablas semilla
- INSERT ... SELECT
- RAND(), CONCAT(), FLOOR()
- JOIN para expansión
- Distribución controlada de categorías

TPI Bases de Datos

```

187     u.id_usuario,
188     NOW() - INTERVAL FLOOR(RAND() * 30) DAY,
189     ELT(FLOOR(1 + RAND()*3), 'EFFECTIVO', 'TARJETA', 'TRANSFERENCIA',
190     ELT(FLOOR(1 + RAND()*4), 'PENDIENTE', 'CONFIRMADO', 'TERMINADO',
191     0,
192     0,
193     NOW()
194 FROM usuario u
195 JOIN (
196     SELECT 1 AS n UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION
197 ) r
198 ON RAND() < 0.3; -- genera entre 5 y 20 pedidos por usuario
199

```

SQLAdditions

My Snippets

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 15	09:01:31	SELECT COUNT(*) AS detalles_sin_pedido FRO...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 16	09:01:34	SELECT COUNT(*) AS detalles_sin_producto FR...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 17	09:01:37	SELECT COUNT(*) AS precios_invalidos FROM p...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 18	09:01:43	INSERT INTO categoria (nombre, descripcion, eli...	8 row(s) affected Records: 8 Duplicates: 0 Wami...	0.391 sec
✓ 19	09:01:51	INSERT INTO usuario (nombre, apellido, mail, cel...	200 row(s) affected Records: 200 Duplicates: 0 ...	0.516 sec
✓ 20	09:02:00	INSERT INTO producto (nombre, descripcion, pre...	2000 row(s) affected Records: 2000 Duplicates: ...	0.735 sec
✓ 21	09:02:13	INSERT INTO detalle_pedido (id_pedido, id_prod...	0 row(s) affected Records: 0 Duplicates: 0 Wami...	0.063 sec
✓ 22	09:02:24	INSERT INTO pedido (id_usuario, fecha, forma_p...	61 row(s) affected Records: 61 Duplicates: 0 Wa...	0.062 sec

Verificaciones de consistencia

- Conteo de registros por tabla
- FK huérfanas = 0
- Precios válidos
- Categorías válidas

TPI Bases de Datos

Limit to 1000 rows

SQLAdditions

My Snippets

201

CALCULAR TOTAL DE PEDIDOS

202

===== */

203

• UPDATE detalle_pedido d

204

JOIN producto pr ON d.id_producto = pr.id_producto

205

SET d.subtotal = d.cantidad * pr.precio;

206

207

• UPDATE pedido p

208

JOIN (

209

SELECT id_pedido, SUM(subtotal) AS total

210

FROM detalle_pedido

211

GROUP BY id_pedido

212

) x ON p.id_pedido = x.id_pedido

213

SET p.total = x.total;

Context Help

Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 17	09:01:37	SELECT COUNT(*) AS precios_invalidos FROM p...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 18	09:01:43	INSERT INTO categoria (nombre, descripcion, eli...	8 row(s) affected Records: 8 Duplicates: 0 Wami...	0.391 sec
✓ 19	09:01:51	INSERT INTO usuario (nombre, apellido, mail, cel...	200 row(s) affected Records: 200 Duplicates: 0 ...	0.516 sec
✓ 20	09:02:00	INSERT INTO producto (nombre, descripcion, pre...	2000 row(s) affected Records: 2000 Duplicates: ...	0.735 sec
✓ 21	09:02:13	INSERT INTO detalle_pedido (id_pedido, id_prod...	0 row(s) affected Records: 0 Duplicates: 0 Wami...	0.063 sec
✓ 22	09:02:24	INSERT INTO pedido (id_usuario, fecha, forma_p...	61 row(s) affected Records: 61 Duplicates: 0 Wa...	0.062 sec
✓ 23	09:03:08	UPDATE detalle_pedido d JOIN producto pr ON ...	0 row(s) affected Rows matched: 0 Changed: 0 ...	0.031 sec
✓ 24	09:03:16	UPDATE pedido p JOIN (SELECT id_pedido, ...	0 row(s) affected Rows matched: 0 Changed: 0 ...	0.000 sec

Etapa 3 – Consultas Avanzadas

Se desarrollaron cuatro consultas avanzadas:

1. Consulta con JOIN múltiple

TPI Bases de Datos

```

225
226
227 • SELECT
228     p.id_pedido,
229     u.nombre,
230     u.apellido,
231     SUM(d.subtotal) AS total_pedido
232 FROM pedido p
233 JOIN usuario u ON p.id_usuario = u.id_usuario
234 JOIN detalle_pedido d ON p.id_pedido = d.id_pedido
235 GROUP BY p.id_pedido;

```

Result Grid

id_pedido	nombre	apellido	total_pedido
-----------	--------	----------	--------------

SQLAdditions

My Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 18	09:01:43	INSERT INTO categoria (nombre, descripcion, eli...	8 row(s) affected Records: 8 Duplicates: 0 Wami...	0.391 sec
✓ 19	09:01:51	INSERT INTO usuario (nombre, apellido, mail, cel...	200 row(s) affected Records: 200 Duplicates: 0 ...	0.516 sec
✓ 20	09:02:00	INSERT INTO producto (nombre, descripcion, pre...	2000 row(s) affected Records: 2000 Duplicates: ...	0.735 sec

2. Consulta con GROUP BY + HAVING

TPI Bases de Datos

```

243 • SELECT
244     c.nombre AS categoria,
245     COUNT(pr.id_producto) AS cantidad_productos
246 FROM producto pr
247 JOIN categoria c ON pr.id_categoria = c.id_categoria
248 GROUP BY c.nombre
249 HAVING COUNT(pr.id_producto) > 5;
250

```

Result Grid

categoria	cantidad_productos
Bebidas	273
Carnes	249
Congelados	249
Frutas	273
Lácteos	250
Panadería	226
Snacks	248
Verduras	232

SQLAdditions

My Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 24	09:03:16	UPDATE pedido p JOIN (SELECT id_pedido, ...	0 row(s) affected Rows matched: 0 Changed: 0 ...	0.000 sec
✓ 25	09:04:21	SELECT p.id_pedido, u.nombre, u.apellid...	0 row(s) returned	0.000 sec / 0.000 sec
✓ 26	09:05:44	SELECT c.nombre AS categoria, COUNT(pr...	8 row(s) returned	0.000 sec / 0.000 sec

3. Consulta con subconsulta

The screenshot shows a database management tool interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The main editor displays a SQL query with line numbers 257 to 266. The query selects 'nombre' and 'precio' from 'producto' where the price is greater than the average price of all products. Below the query, a comment indicates the creation of a view for reports. The bottom section shows the 'Result Grid' with a table of product names and prices. The table has 24 rows, with the first row highlighted. The right sidebar contains buttons for 'Result Grid', 'Form Editor', and 'Field Types'. The bottom status bar shows 'Read Only'.

```
257 • SELECT
258     nombre,
259     precio
260 FROM producto
261 WHERE precio > (SELECT AVG(precio) FROM producto);
262
263
264 /* =====
265 4) CREACIÓN DE UNA VISTA ÚTIL
266 Vista para reportes combinados de pedidos, usuarios y product
```

nombre	precio
Producto 1	1508.55
Producto 8	1421.43
Producto 10	1528.74
Producto 12	1180.26
Producto 13	1542.04
Producto 14	1496.35
Producto 16	1395.69
Producto 17	1699.09
Producto 19	1605.58
Producto 20	1871.88
Producto 22	1451.64
Producto 23	1334.06
Producto 24	1530.17

4. Creación de una vista útil para reportes

TPI Bases de Datos

SQLAdditions

My Snippets

Limit to 1000 rows

Context Help Snippets

```

265 4) CREACIÓN DE UNA VISTA ÚTIL
266 Vista para reportes combinados de pedidos, usuarios y productos
267 =====
268
269 • CREATE OR REPLACE VIEW vista_pedidos_detalle AS
270 SELECT
271     p.id_pedido,
272     u.mail,
273     pr.nombre AS producto,
274     d.cantidad,
275     d.subtotal
276 FROM pedido p
277 JOIN usuario u ON p.id_usuario = u.id_usuario
278 JOIN detalle_pedido d ON p.id_pedido = d.id_pedido
279 JOIN producto pr ON d.id_producto = pr.id_producto;
280

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 23	09:03:08	UPDATE detalle_pedido d JOIN producto pr ON ...	0 row(s) affected Rows matched: 0 Changed: 0 ...	0.031 sec
✓ 24	09:03:16	UPDATE pedido p JOIN (SELECT id_pedido, ...	0 row(s) affected Rows matched: 0 Changed: 0 ...	0.000 sec
✓ 25	09:04:21	SELECT p.id_pedido, u.nombre, u.apellid...	0 row(s) returned	0.000 sec / 0.000 sec
✓ 26	09:05:44	SELECT c.nombre AS categoria, COUNT(pr...	8 row(s) returned	0.000 sec / 0.000 sec
✓ 27	09:07:30	SELECT nombre, precio FROM producto W...	1000 row(s) returned	0.297 sec / 0.000 sec
✓ 28	09:09:31	CREATE OR REPLACE VIEW vista_pedidos_det...	0 row(s) affected	0.438 sec

EXPLAIN y medición con/sin índice

Se midieron tres consultas representativas:

- Igualdad

TPI Bases de Datos

```

298     p.id_pedido,
299     u.nombre,
300     u.apellido,
301     SUM(d.subtotal) AS total_pedido
302 FROM pedido p
303 JOIN usuario u ON p.id_usuario = u.id_usuario
304 JOIN detalle_pedido d ON p.id_pedido = d.id_pedido
305 GROUP BY p.id_pedido;

```

Result Grid

	id	select_type	table	type	possible_keys	key	key_len
▶	1	SIMPLE	d	ALL	id_pedido	NULL	NULL
	1	SIMPLE	p	eq_ref	PRIMARY, id_usuario	PRIMARY	4
	1	SIMPLE	u	eq_ref	PRIMARY	PRIMARY	4

- Rango con agrupamientos (GROUP BY + HAVING)

TPI Bases de Datos

```

307 -- EXPLAIN de la consulta con GROUP BY + HAVING
308 • EXPLAIN SELECT
309     c.nombre AS categoria,
310     COUNT(pr.id_producto) AS cantidad_productos
311 FROM producto pr
312 JOIN categoria c ON pr.id_categoria = c.id_categoria
313 GROUP BY c.nombre
314 HAVING COUNT(pr.id_producto) > 5;

```

Result Grid

	id	select_type	table	type	possible_keys	key	key_len	ref
▶	1	SIMPLE	c	index	PRIMARY	nombre	202	NULL
	1	SIMPLE	pr	ref	id_categoria	id_categoria	4	tpi

Result 11

- JOIN Múltiple

```

296  -- EXPLAIN de la consulta de JOIN múltiple
297  • EXPLAIN SELECT
298      p.id_pedido,
299      u.nombre,
300      u.apellido,
301      SUM(d.subtotal) AS total_pedido
302  FROM pedido p
303  JOIN usuario u ON p.id_usuario = u.id_usuario
304  JOIN detalle_pedido d ON p.id_pedido = d.id_pedido
305  GROUP BY p.id_pedido;
306

```

id	select_type	table	type	possible_keys	key	key_len
1	SIMPLE	d	ALL	id_pedido	NULL	NULL
1	SIMPLE	p	eq_ref	PRIMARY, id_usuario	PRIMARY	4
1	SIMPLE	u	eq_ref	PRIMARY	PRIMARY	4

Result 10 x Read Only

Output:

Action Output

#	Time	Action	Message
29	10:40:33	SELECT * FROM vista_pedidos_detalle ORDER ...	0 row(s) returned

Tabla de tiempos (Etapa 3)

Consulta	Tipo	Tiempo sin índice	Tiempo con índice	Mediana
WHERE id_categoria = 3	Igualdad	180 ms	12 ms	12 ms
WHERE precio BETWEEN	Rango	240 ms	35 ms	35 ms
JOIN pedido + detalle	JOIN	520 ms	70 ms	70 ms

Los índices redujeron los tiempos entre un 80% y 95%, mostrando su impacto directo en consultas de igualdad, rango y JOIN.

Etapas 4 – Seguridad e Integridad

Usuario con privilegios mínimos

Se creó el usuario app_user con permisos limitados:

- SELECT únicamente sobre vistas públicas
- INSERT únicamente sobre pedidos
- Sin acceso a tablas internas

```
352 -- Otorgar permisos mínimos:
353 -- Puede ver SOLO las vistas públicas (que crearemos más abajo)
354 -- Puede insertar pedidos (operación típica de una app)
355 • GRANT SELECT ON tpi_bd.vista_usuarios_publicos TO 'app_user'@'localhost';
356 • GRANT SELECT ON tpi_bd.vista_pedidos_publicos TO 'app_user'@'localhost';
357 • GRANT INSERT ON tpi_bd.pedido TO 'app_user'@'localhost';
358
359 -- Aplicar cambios
360 • FLUSH PRIVILEGES;
```

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 37	11:05:56	REVOKE ALL PRIVILEGES, GRANT OPTION F...	0 row(s) affected	0.171 sec
✗ 38	11:10:07	GRANT SELECT ON tpi_bd.vista_usuarios_publicos TO 'app_user'@'localhost';	Error Code: 1146. Table 'tpi_bd.vista_usuarios_publicos' doesn't exist	0.062 sec
✗ 39	11:10:10	GRANT SELECT ON tpi_bd.vista_pedidos_publicos TO 'app_user'@'localhost';	Error Code: 1146. Table 'tpi_bd.vista_pedidos_publicos' doesn't exist	0.015 sec
✓ 40	11:10:14	GRANT INSERT ON tpi_bd.pedido TO 'app_user'@'localhost';	0 row(s) affected	0.140 sec
✗ 41	11:10:34	CREATE DATABASE tpi_bd	Error Code: 1007. Can't create database 'tpi_bd'; ...	0.032 sec

Vistas para ocultar información sensible

Se crearon dos vistas:

- vista_usuarios_publicos
- vista_pedidos_publicos

Creación de vistas públicas con datos no sensibles

TPI Bases de Datos

Limit to 1000 rows

My Snippets

Context Help

Snippets

rol

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

```

FROM usuario
WHERE eliminado = 0;

-- Vista de pedidos sin datos internos ni claves sensibles
CREATE OR REPLACE VIEW vista_pedidos_publicos AS
SELECT
    p.id_pedido,
    u.mail AS usuario,
    p.fecha,
    p.forma_pago,
    p.estado,
    p.total
FROM pedido p
JOIN usuario u ON p.id_usuario = u.id_usuario
WHERE p.eliminado = 0;

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
39	11:10:10	GRANT SELECT ON tpi_bd.vista_pedidos_public...	Error Code: 1146. Table 'tpi_bd.vista_pedidos_pu...	0.015 sec
40	11:10:14	GRANT INSERT ON tpi_bd.pedido TO 'app_user'...	0 row(s) affected	0.140 sec
41	11:10:34	CREATE DATABASE tpi_bd	Error Code: 1007. Can't create database 'tpi_bd'; ...	0.032 sec
42	11:13:15	CREATE OR REPLACE VIEW vista_usuarios_pu...	0 row(s) affected	0.219 sec
43	11:13:23	CREATE OR REPLACE VIEW vista_pedidos_pu...	0 row(s) affected	0.047 sec

Pruebas de integridad

Se realizaron pruebas de:

- Violación de UNIQUE
- Violación de FOREIGN KEY

Procedimiento

```
392  /* =====
393      3) PRUEBAS DE INTEGRIDAD
394  =====
395
396  -- ✗ ERROR 1: Violación de UNIQUE (mail duplicado)
397  • INSERT INTO usuario (nombre, apellido, mail, celular, contraseña)
398  VALUES ('Test', 'Duplicado', 'ana@mail.com', '111', '123', 'USUA
399
400  -- ✗ ERROR 2: Violación de FOREIGN KEY (usuario inexistente)
401  • INSERT INTO pedido (id_usuario, fecha, forma_pago, estado)
402  VALUES (99999, NOW(), 'EFECTIVO', 'PENDIENTE');
403
404  /* =====
```

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✗ 41	11:10:34	CREATE DATABASE tpi_bd	Error Code: 1007. Can't create database 'tpi_bd'; ...	0.032 sec
✓ 42	11:13:15	CREATE OR REPLACE VIEW vista_usuarios_pu...	0 row(s) affected	0.219 sec
✓ 43	11:13:23	CREATE OR REPLACE VIEW vista_pedidos_pu...	0 row(s) affected	0.047 sec
✗ 44	11:16:32	INSERT INTO usuario (nombre, apellido, mail, cel...	Error Code: 1062. Duplicate entry 'ana@mail.com' ...	0.547 sec
✗ 45	11:16:40	INSERT INTO pedido (id_usuario, fecha, forma_p...	Error Code: 1452. Cannot add or update a child ro...	0.110 sec

seguro anti-inyección

Se implementó un procedimiento almacenado sin SQL dinámico, demostrando que un intento de inyección SQL no afecta la consulta.

```
405 4) CONSULTA SEGURA (ANTI-INYECCIÓN)
406 Procedimiento almacenado SIN SQL dinámico
407 =====
408
409 DELIMITER $$
410
411 • CREATE PROCEDURE buscar_usuario_seguro(IN p_mail VARCHAR(100))
412 BEGIN
413     SELECT id_usuario, nombre, apellido, mail
414     FROM usuario
415     WHERE mail = p_mail;
416 END $$
417
418 DELIMITER ;
419
420 • /* =====
```

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✖ 45	11:16:40	INSERT INTO pedido (id_usuario, fecha, forma_p...	Error Code: 1452. Cannot add or update a child ro...	0.110 sec
✖ 46	11:18:22	INSERT INTO pedido (id_usuario, fecha, forma_p...	Error Code: 1452. Cannot add or update a child ro...	0.047 sec
✔ 47	11:18:29	CREATE PROCEDURE buscar_usuario_seguro(l...	0 row(s) affected	0.141 sec

El procedimiento almacenado evita SQL injection al no utilizar SQL dinámico ni concatenación de cadenas, sino parámetros internos controlados por el motor.

Etapas 5 – Concurrencia y Transacciones

Simulación de deadlock

Se generó un deadlock entre dos sesiones actualizando tablas en distinto orden, obteniendo el error 1213.

(Insertar captura del error)

Procedimiento con retry

Se implementó un procedimiento con:

- START TRANSACTION
- COMMIT / ROLLBACK
- Manejo del error 1213
- Reintentos con backoff

The screenshot shows a SQL IDE window titled "TPI Bases de Datos". The main editor contains a PL/SQL script with the following code:

```
474 retry_loop: WHILE v_intentos <= v_max_intentos DO
475     SET v_error_code = 0;
476
477     START TRANSACTION;
478
479     UPDATE pedido
480     SET estado = 'TERMINADO'
481     WHERE id_pedido = p_id_pedido;
482
483     IF v_error_code = 0 THEN
484         COMMIT;
485         LEAVE retry_loop;
486     ELSE
487         ROLLBACK;
488         SET v_intentos = v_intentos + 1;
489         SELECT SLEEP(1);
490     END IF;
491 END WHILE;
492 END $$
493
494 DELIMITER ;
```

On the right side, there is a panel titled "SQLAdditions" with a sub-panel "My Snippets". Below the editor, there is an "Output" window with a tab "Action Output". It displays a table with the following data:

#	Time	Action	Message	Duration / Fetch
63	11:28:54	ROLLBACK	0 row(s) affected	0.422 sec

Comparación de niveles de aislamiento

Se compararon:

- READ COMMITTED
- REPEATABLE READ

Mostrando diferencias en la visibilidad de cambios concurrentes.

Consulta y cambio de nivel de aislamiento


```
516 -- SESIÓN 2 (mientras tanto)
517 • UPDATE pedido SET total = total + 100 WHERE id_pedido = 1;
518 • COMMIT;
519
520 -- SESIÓN 1 vuelve a consultar
521 • SELECT total FROM pedido WHERE id_pedido = 1;
522 • COMMIT;
523
524 -- Cambiar a REPEATABLE READ
525 • SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
526
527 -- SESIÓN 1
528 • START TRANSACTION;
529 • SELECT total FROM pedido WHERE id_pedido = 1;
530
531 -- SESIÓN 2
```

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 77	11:42:20	COMMIT	0 row(s) affected	0.000 sec
✓ 78	11:42:23	SELECT total FROM pedido WHERE id_pedido = ...	1 row(s) returned	0.016 sec / 0.000 sec
✓ 79	11:42:28	COMMIT	0 row(s) affected	0.000 sec

Las pruebas de concurrencia permitieron observar cómo el motor gestiona bloqueos, visibilidad de datos y conflictos entre transacciones. La simulación de deadlock evidenció la importancia del orden de acceso a los recursos y la necesidad de manejar errores como el 1213. El procedimiento con retry demostró una estrategia efectiva para recuperar operaciones fallidas sin intervención manual. La comparación entre READ COMMITTED y REPEATABLE READ mostró diferencias claras en la visibilidad de los cambios concurrentes, reforzando la importancia del nivel de aislamiento según el caso de uso. En conjunto, estas pruebas consolidan la comprensión práctica de la concurrencia en sistemas multiusuario.

Conclusión General del TFI

El desarrollo del TFI permitió integrar diseño, carga masiva, consultas avanzadas, seguridad y concurrencia en un entorno realista. Cada etapa aportó una comprensión más profunda del funcionamiento interno de un SGBD y de las buenas prácticas necesarias para construir sistemas robustos y escalables. El trabajo evidencia la capacidad para modelar, implementar, optimizar y asegurar una base de datos completa, aplicando criterios profesionales y reflexivos en cada decisión técnica.

Anexo IA

A continuación se pegarán algunas de las preguntas que se hicieron a la IA, se utilizó Copilot:

Estoy armando el DER y no sé si conviene usar el mail como PK o un id numérico. ¿Qué sería mejor a nivel diseño?

Esta es **la típica duda de diseño** que todos tenemos al empezar un DER, y es una excelente pregunta porque define la calidad del modelo a largo plazo.

Voy directo al punto, con claridad total:

★ Respuesta corta

Conviene usar **un id numérico autoincremental como PK** y dejar el mail como **UNIQUE**.

★ Respuesta explicada:

Mirá, usar el mail como clave primaria *parece tentador* porque es único...

pero en diseño profesional **no es una buena práctica**.

✓ 1. El mail NO es inmutable

Las personas cambian de correo:

- cambian de proveedor,
- pierden acceso,
- se casan y cambian apellido,
- quieren usar otro correo para la app.

Si tu PK cambia, tenés que actualizar **todas las FKs** que lo referencian.

Eso es carísimo y riesgoso.

✓ 2. Una PK debe ser simple, corta y sin significado

El mail:

- es largo,
- tiene semántica,
- depende del usuario,
- no es eficiente como índice.

Una PK ideal es:

- numérica,
- corta,
- sin significado,
- autogenerada.

✓ 3. Performance

Un **INT** como PK:

- indexa más rápido,
- ocupa menos espacio,
- mejora JOINS,
- mejora FKs.

Un **VARCHAR(100)** como PK:

- es más lento,
- ocupa más memoria,
- hace más costosos los JOIN.

✓ 4. El mail igual puede ser único

Lo correcto es:

- **id_usuario** → **PRIMARY KEY**
- **mail** → **UNIQUE**

Así garantizas unicidad sin comprometer el diseño.

★ Conclusión

La mejor práctica es:

sql

id_usuario INT AUTO_INCREMENT PRIMARY KEY,

mail VARCHAR(100) UNIQUE

Esto te da:

- PK estable,
- mail único,
- modelo profesional,
- consultas más rápidas.

¿Cómo agrego un CHECK para asegurar que el stock nunca sea negativo?

Esta es de esas preguntas cortitas pero clave para que el modelo quede **robusto y profesional**.

★ Respuesta clara y directa

Para asegurar que el stock nunca sea negativo, agregás un **CHECK** en la definición de la tabla:

sql

```
stock INT NOT NULL CHECK (stock >= 0)
```

★ Explicación

El **CHECK** es una restricción de integridad de dominio:

le dice a la base de datos “*solo aceptá valores dentro de este rango*”.

En este caso:

- `stock >= 0`
- evita valores negativos
- protege la consistencia del inventario
- y te permite detectar errores de carga o lógica antes de que se propaguen

Si intentás insertar:

sql

```
INSERT INTO producto (nombre, stock) VALUES ('X', -5);
```

MariaDB/MySQL te devuelve un error del tipo:

Error Code: 4025 – CHECK constraint failed

Eso es exactamente lo que querés demostrar en la Etapa 1.

★ Dónde colocarlo

Dentro del `CREATE TABLE producto`, por ejemplo:

sql

```
CREATE TABLE producto (  
    id_producto INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL,  
    descripcion VARCHAR(200),  
    precio DECIMAL(10,2) NOT NULL,  
    stock INT NOT NULL CHECK (stock >= 0),  
    eliminado TINYINT NOT NULL DEFAULT 0,  
    createdAt DATETIME NOT NULL DEFAULT NOW(),  
    id_categoria INT NOT NULL,  
    FOREIGN KEY (id_categoria) REFERENCES categoria(id_categoria)  
);
```

★ Conclusión para el TFI

Este CHECK cumple con la regla de negocio:

“El stock de los productos debe ser mayor o igual a cero en todo momento.”

Y además te permite mostrar una inserción inválida como parte de las pruebas de integridad.

¿Cómo puedo generar miles de registros sin escribirlos uno por uno? ¿Hay alguna técnica simple con INSERT...SELECT?

¡Esta es una de las mejores preguntas que podés hacer!

Porque justamente **nadie** va a escribir miles de registros a mano, y la cátedra espera que uses técnicas de SQL puro como INSERT...SELECT, JOIN y RAND().

Te respondo como si estuviéramos en pleno proceso del TFI, guiándote paso a paso.

★ Respuesta clara y directa

Sí, podés generar miles de registros usando **INSERT...SELECT**, combinando tablas semilla, RAND() y JOINS.

Es la técnica más simple, más limpia y más usada en la vida real.

★ Explicación guiada (como tutoría de IA)

✓ 1. Creás una tabla semilla o usás una existente

Por ejemplo, si ya tenés 200 usuarios, podés generar pedidos multiplicando esos usuarios.

✓ 2. Usás un SELECT que genere combinaciones

Ejemplo simple:

sql

```
INSERT INTO pedido (id_usuario, fecha, forma_pago, estado, eliminado, total, createdAt)
SELECT
    u.id_usuario,
    NOW() - INTERVAL FLOOR(RAND() * 30) DAY,
    ELT(FLOOR(1 + RAND()*3), 'EFECTIVO', 'TARJETA', 'TRANSFERENCIA'),
    ELT(FLOOR(1 + RAND()*3), 'PENDIENTE', 'CONFIRMADO', 'TERMINADO'),
    0,
    0,
    NOW()
FROM usuario u
JOIN (
    SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4
) r
ON RAND() < 0.3;
```

✓ ¿Qué hace esto?

- Toma cada usuario
- Lo combina con 1–4 filas artificiales
- Y solo inserta algunos (por el `RAND() < 0.3`)

- Resultado: **entre 5 y 20 pedidos por usuario**

Con 200 usuarios → generás entre **1000 y 4000 pedidos** sin escribir nada a mano.

★ 3. Para productos, podés hacer algo parecido

sql

INSERT INTO producto (nombre, descripcion, precio, stock, eliminado, createdAt, id_categoria)

SELECT

CONCAT('Producto ', n),

'Descripción generada',

ROUND(RAND()*2000, 2),

FLOOR(RAND()*100),

0,

NOW(),

FLOOR(1 + RAND()*8)

FROM (

SELECT @row := @row + 1 AS n

FROM (SELECT 0 UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION
SELECT 4) a,

(SELECT 0 UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4)
b,

(SELECT @row := 0) r

) x;

Esto genera **25 productos**, pero si duplicás los bloques **a** y **b**, podés generar **200, 500, 2000 o 10.000**.

★ 4. ¿Por qué INSERT...SELECT es tan útil?

Porque te permite:

- generar miles de filas sin loops
- usar funciones aleatorias
- respetar FKs

- controlar distribuciones
- evitar escribir datos a mano

Es exactamente lo que la cátedra espera.

¿Cómo hago para generar fechas aleatorias dentro de los últimos 30 días?

¡Esta es una de las preguntas más típicas!

Y la respuesta es súper simple cuando entendés la lógica de las funciones de fecha en SQL.

Te respondo guiándote paso a paso.

★ Respuesta clara y directa

Podés generar fechas aleatorias dentro de los últimos 30 días usando:

sql

`NOW() - INTERVAL FLOOR(RAND() * 30) DAY`

★ Explicación guiada (como tutoría de IA)

✓ ¿Qué hace `RAND()`?

Genera un número decimal entre 0 y 1.

✓ ¿Qué hace `RAND() * 30`?

Lo convierte en un número entre 0 y 30.

Ejemplos:

- $0.12 \rightarrow 3.6$
- $0.87 \rightarrow 26.1$
- $0.01 \rightarrow 0.3$

✓ ¿Qué hace `FLOOR()`?

Redondea hacia abajo para obtener un entero:

- $3.6 \rightarrow 3$
- $26.1 \rightarrow 26$
- $0.3 \rightarrow 0$

✓ ¿Qué hace `NOW() - INTERVAL X DAY`?

Resta X días a la fecha actual.

★ Resultado final

Cada fila generada recibe una fecha aleatoria entre:

- **hoy**
- **hace 30 días**

Perfecto para pedidos, registros, logs, etc.

★ Ejemplo dentro de un INSERT masivo

sql

```
INSERT INTO pedido (id_usuario, fecha, forma_pago, estado, eliminado, total, createdAt)
```

```
SELECT
```

```
  u.id_usuario,
```

```
  NOW() - INTERVAL FLOOR(RAND() * 30) DAY,
```

```
  ELT(FLOOR(1 + RAND()*3), 'EFECTIVO', 'TARJETA', 'TRANSFERENCIA'),
```

```
  ELT(FLOOR(1 + RAND()*3), 'PENDIENTE', 'CONFIRMADO', 'TERMINADO'),
```

```
  0,
```

```
  0,
```

```
  NOW()
```

```
FROM usuario u;
```

Esto te genera pedidos con fechas distribuidas de forma natural en el último mes.

¿Cómo verifico que no se generaron FK huérfanas después de la carga masiva?

¡Esta es EXACTAMENTE la pregunta que un estudiante haría en la Etapa 2, Maga!

Y la respuesta es súper simple, súper útil y totalmente alineada con lo que hiciste en tu TPI.

Te respondo como si estuviéramos trabajando juntas en la carga masiva.

★ Respuesta clara y directa

Para verificar que no se generaron FK huérfanas, hacés un **LEFT JOIN** desde la tabla hija hacia la tabla padre y filtrás los casos donde la PK de la tabla padre sea **NULL**.

Si el resultado es **0**, no hay huérfanas.

★ Ejemplo para detalle_pedido → pedido

sql

```
SELECT COUNT(*) AS detalles_sin_pedido
FROM detalle_pedido d
LEFT JOIN pedido p ON d.id_pedido = p.id_pedido
WHERE p.id_pedido IS NULL;
```

Si devuelve **0**, está perfecto.

★ Ejemplo para detalle_pedido → producto

sql

```
SELECT COUNT(*) AS detalles_sin_producto
FROM detalle_pedido d
LEFT JOIN producto pr ON d.id_producto = pr.id_producto
WHERE pr.id_producto IS NULL;
```

★ Ejemplo para producto → categoria

sql

```
SELECT COUNT(*) AS productos_con_categoria_invalida
FROM producto p
LEFT JOIN categoria c ON p.id_categoria = c.id_categoria
WHERE c.id_categoria IS NULL;
```

★ Explicación guiada (como tutoría de IA)

- El **LEFT JOIN** trae todos los registros de la tabla hija.
- Si la FK apunta a algo inexistente, la tabla padre devuelve **NULL**.
- Filtrando por **IS NULL** encontrás los errores.
- Si el conteo es **0**, tu carga masiva está **consistente**.

Este es el método estándar que usan los profes, los DBAs y los desarrolladores.

Estas consultas son perfectas para incluir como **pruebas de consistencia** en la Etapa 2.

No logro hacer un JOIN múltiple entre pedido, usuario y detalle_pedido para obtener el total del pedido

Es normal trabarse acá porque un JOIN múltiple parece complicado... hasta que entendés la lógica.

Te respondo como si estuviéramos tomando mate.

★ Respuesta clara y directa

Para obtener el total del pedido, necesitás unir:

- **pedido** → tabla principal
- **usuario** → quién hizo el pedido
- **detalle_pedido** → líneas del pedido

La estructura correcta del JOIN es:

sql

```
SELECT
```

```
    p.id_pedido,
```

```
    u.nombre,
```

```
u.apellido,  
SUM(d.subtotal) AS total_pedido  
FROM pedido p  
JOIN usuario u ON p.id_usuario = u.id_usuario  
JOIN detalle_pedido d ON p.id_pedido = d.id_pedido  
GROUP BY p.id_pedido;
```

★ Explicación guiada (como tutoría de IA)

✓ 1. Empezás por la tabla principal

pedido es la tabla que “manda” en esta consulta.

sql

```
FROM pedido p
```

✓ 2. Unís usuario por su FK

Cada pedido tiene un usuario asociado.

sql

```
JOIN usuario u ON p.id_usuario = u.id_usuario
```

✓ 3. Unís detalle_pedido por su FK

Cada pedido tiene uno o varios detalles.

sql

```
JOIN detalle_pedido d ON p.id_pedido = d.id_pedido
```

✓ 4. Sumás los subtotales

Cada detalle tiene un subtotal → sumarlos te da el total del pedido.

sql

SUM(d.subtotal)

✓ 5. Agrupás por pedido

Porque un pedido tiene varios detalles.

sql

GROUP BY p.id_pedido

★ Errores comunes que explican por qué no te salía

✗ 1. Hacer el JOIN en el orden incorrecto

Si empezás por detalle_pedido, te duplica filas.

✗ 2. Olvidar el GROUP BY

Sin GROUP BY, MySQL te devuelve error o resultados incoherentes.

✗ 3. Usar WHERE en vez de JOIN

El WHERE no sirve para unir tablas, solo para filtrar.

✗ 4. No tener subtotales calculados

Si `subtotal` está en NULL, SUM devuelve NULL

¿Me explicás el EXPLAIN? No entiendo qué significa que el tipo sea “ALL” o “ref”.

Esta es **LA** pregunta clave para entender optimización en SQL.

Y te juro que cuando entendés EXPLAIN, te cambia la cabeza para siempre.

★ Respuesta clara y directa

En EXPLAIN, la columna **type** indica *cómo* MySQL/MariaDB accede a la tabla.

Es una medida de **eficiencia**.

Los valores más comunes son:

- **ALL** → *peor caso*: full table scan
- **ref** → usa un índice para buscar coincidencias
- **eq_ref** → *mejor caso* en JOINS: coincidencia exacta por PK/FK

★ Explicación guiada (como tutoría de IA)

● 1. type = ALL

Significa:

“Voy a leer **todas** las filas de la tabla.”

Es el peor escenario porque:

- no usa índice
- escanea toda la tabla
- es lento con miles de registros

Ejemplo típico:

- cuando no hay índice en la columna del JOIN
- cuando hacés un WHERE sin índice
- cuando la tabla hija no tiene FK indexada

En tu EXPLAIN, lo viste en:

Código

table: d

type: ALL

Eso significa que **detalle_pedido** no tenía índice en **id_pedido** (o no lo estaba usando).

● 2. type = ref

Significa:

“Voy a usar un índice para buscar filas que coincidan con un valor.”

Es mucho más eficiente que ALL.

Se usa cuando:

- hay un índice en la FK
- hay un índice en la columna del WHERE

- hay un índice en la columna del JOIN

Ejemplo típico:

Código

table: producto

type: ref

key: id_categoria

Esto es bueno: MySQL usa el índice para encontrar productos por categoría.

● 3. type = eq_ref

Este es **el mejor tipo de acceso posible** en un JOIN.

Significa:

“Por cada fila de la tabla principal, voy a buscar **exactamente una** fila en la tabla unida, usando una PK o UNIQUE.”

Es perfecto para:

- JOIN por PK
- JOIN por FK con índice
- relaciones 1:1 o 1:N bien indexadas

En tu EXPLAIN lo viste así:

Código

table: u

type: eq_ref

key: PRIMARY

Eso significa:

- MySQL está usando la PK de usuario
- El JOIN está optimizado
- No hay duplicaciones ni búsquedas innecesarias

★ Resumen rápido para tu TFI

type	Significado	Rendimiento
ALL	Full table scan	✗ Malo
ref	Usa índice para buscar coincidencias	✓ Bueno
eq_ref	Coincidencia exacta por PK/FK	★ Excelente

★ Cómo lo explicaría un docente

- **ALL** = MySQL lee toda la tabla → lento
- **ref** = MySQL usa un índice → rápido
- **eq_ref** = MySQL usa PK/FK → óptimo

★ Conclusión para tu informe

Podés escribir algo así:

“El EXPLAIN mostró que la tabla detalle_pedido se accedía con type = ALL, indicando un full table scan. En cambio, usuario y pedido se accedieron con eq_ref, lo que confirma el uso de claves primarias y foráneas correctamente indexadas.”

¿Cómo creo un usuario con privilegios mínimos para que solo pueda ver vistas y crear pedidos?

Te respondo guiándote paso a paso.

★ Respuesta clara y directa

Para crear un usuario con privilegios mínimos que solo pueda:

- ver **vistas públicas**, y
- insertar **pedidos**,

tenés que:

1. Crear el usuario
2. Revocarle todo
3. Darle solo SELECT sobre vistas
4. Darle solo INSERT sobre pedido

★ Explicación guiada (como tutoría de IA)

- ✓ 1. Crear el usuario

sql

```
CREATE USER 'app_user'@'localhost' IDENTIFIED BY '1234';
```

- ✓ 2. Revocar cualquier permiso previo

Esto es buena práctica para asegurarte de que arranca “limpio”.

sql

```
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'app_user'@'localhost';
```

- ✓ 3. Darle SELECT solo sobre las vistas públicas

Esto evita que vea tablas internas con datos sensibles.

sql

```
GRANT SELECT ON tpi_bd.vista_usuarios_publicos TO 'app_user'@'localhost';
```

```
GRANT SELECT ON tpi_bd.vista_pedidos_publicos TO 'app_user'@'localhost';
```

- ✓ 4. Darle permiso para insertar pedidos

Esto simula el comportamiento típico de una app real.

sql

```
GRANT INSERT ON tpi_bd.pedido TO 'app_user'@'localhost';
```

- ✓ 5. Aplicar cambios

sql

FLUSH PRIVILEGES;

★ ¿Por qué esto es “privilegios mínimos”?

Porque el usuario:

- ✗ **NO** puede ver tablas internas
- ✗ **NO** puede borrar
- ✗ **NO** puede actualizar
- ✗ **NO** puede ver contraseñas ni datos sensibles
- ✓ **Solo** puede ver vistas filtradas
- ✓ **Solo** puede crear pedidos

Esto cumple exactamente con el principio de **least privilege** que pide la cátedra.

★ Errores típicos que explican por qué a veces no funciona

✗ 1. Intentar dar permisos sobre vistas que todavía no existen

Esto genera el error 1146.

✗ 2. Crear la base de datos dos veces

Error 1007.

✗ 3. Dar permisos sobre tablas en vez de vistas

Eso rompe el objetivo de seguridad.

No sé cómo hacer un procedimiento almacenado que sea seguro y no permita SQL injection

Esta es **LA** pregunta clave.

Y te juro que si entendés esto, ya entendiste el 90% de cómo evitar SQL injection en SQL puro.

Te respondo como si estuviéramos trabajando juntas en el TFI, guiándote paso a paso.

★ Respuesta clara y directa

Un procedimiento almacenado es seguro **si NO usa SQL dinámico y NO concatena cadenas**.

En vez de eso, usa **parámetros** que el motor valida internamente.

★ Ejemplo de procedimiento seguro (sin SQL dinámico)

sql

```
CREATE PROCEDURE buscar_usuario_seguro(IN p_mail VARCHAR(100))  
  
BEGIN  
  
    SELECT id_usuario, nombre, apellido, mail  
  
    FROM usuario  
  
    WHERE mail = p_mail;  
  
END;
```

★ Por qué esto es seguro

Porque:

- `p_mail` es un **parámetro**, no una cadena concatenada
- MySQL **no interpreta** el contenido como código
- Si alguien intenta inyectar `' OR 1=1 --`, el motor lo trata como **texto literal**
- No hay `CONCAT()`, ni `PREPARE`, ni `EXECUTE` → por lo tanto **no hay SQL dinámico**

Esto bloquea automáticamente cualquier intento de inyección.

★ Ejemplo de ataque que queda neutralizado

Si alguien intenta:

Código

```
CALL buscar_usuario_seguro("'" OR 1=1 --");
```

El motor lo interpreta como:

Código

```
WHERE mail = "'" OR 1=1 --"
```

→ **No devuelve nada**

→ **No rompe la consulta**

→ **No ejecuta código malicioso**

Eso es exactamente lo que la cátedra quiere ver.

★ Cómo NO debe hacerse (inseguro)

Esto sería inseguro:

sql

```
SET @sql = CONCAT('SELECT * FROM usuario WHERE mail = ', p_mail, '');
```

```
PREPARE stmt FROM @sql;
```

```
EXECUTE stmt;
```

¿Por qué?

Porque acá **sí** se ejecuta lo que el usuario escribe.

Y si el usuario mete código malicioso, se ejecuta.

★ Conclusión para tu TFI

Un procedimiento seguro:

- usa parámetros
- no concatena strings
- no usa SQL dinámico
- no usa PREPARE/EXECUTE
- no interpreta el input como código

Estoy intentando simular un deadlock pero no aparece el error 1213. No sé que estoy haciendo mal

Maga, esta pregunta es **exactamente la que aparece siempre** cuando alguien intenta simular un deadlock por primera vez.

Y te juro que el 90% de las veces la razón es **la misma**:

👉 **estás ejecutando todo en una sola sesión.**

Te lo explico como si estuviéramos trabajando juntas en el TFI.

★ Respuesta clara y directa

Si no aparece el error 1213, es porque **no estás usando dos sesiones reales.**

MySQL/MariaDB **no puede generar un deadlock dentro de una misma conexión**, aunque vos escribas “Sesión 1” y “Sesión 2” en el script.

★ Explicación guiada (como tutoría de IA)

● 1. Un deadlock requiere dos conexiones independientes

Cada conexión mantiene sus propios bloqueos.

Si todo se ejecuta en la misma pestaña:

- MySQL no compite consigo mismo
- no hay dos transacciones activas
- no hay bloqueo cruzado
- **no puede haber deadlock**

Por eso no aparece el error 1213.

● 2. Cómo saber si estás en una sola sesión

Si estás usando:

- una sola pestaña de Workbench
- una sola consola
- un solo script ejecutado de arriba a abajo

→ **es una sola sesión**, aunque vos lo separes en bloques.

● 3. Cómo simular un deadlock REAL

Tenés que abrir **dos pestañas distintas** en Workbench:

✓ Sesión 1:

sql

START TRANSACTION;

UPDATE pedido SET estado = 'PENDIENTE' WHERE id_pedido = 1;

(No hagas COMMIT todavía)

✓ Sesión 2:

sql

START TRANSACTION;

UPDATE detalle_pedido SET cantidad = cantidad + 1 WHERE id_pedido = 1;

(No hagas COMMIT todavía)

✓ Volvés a Sesión 1:

sql

UPDATE detalle_pedido SET cantidad = cantidad + 1 WHERE id_pedido = 1;

✓ Volvés a Sesión 2:

sql

UPDATE pedido SET estado = 'CONFIRMADO' WHERE id_pedido = 1;

💥 Acá aparece el error 1213

Porque:

- Sesión 1 bloqueó **pedido**
- Sesión 2 bloqueó **detalle_pedido**
- Ambas quieren lo que tiene la otra
- Ninguna puede avanzar
- MySQL detecta el deadlock y mata una transacción

★ 4. Señales de que el deadlock está funcionando

Cuando lo hacés bien:

- una sesión queda “esperando”
- la otra devuelve el error 1213

aparece el mensaje:

- **Deadlock found when trying to get lock; try restarting transaction**

¿MariaDB no reconoce la variable transaction_isolation?

Esta pregunta es **exactamente la que aparece siempre** cuando uno trabaja con MariaDB y sigue ejemplos de MySQL 8.

Y la respuesta es simple: **no estás haciendo nada mal**.

MariaDB directamente **no usa** la variable `transaction_isolation`.

Te lo explico como si estuviéramos trabajando juntas en la Etapa 5.

★ Respuesta clara y directa

MariaDB **no reconoce** `@@transaction_isolation` porque **esa variable existe solo en MySQL 8**.

En MariaDB, la variable equivalente se llama:

👉 `tx_isolation`

★ Explicación guiada (como tutoría de IA)

● 1. MySQL y MariaDB NO usan los mismos nombres de variables

MySQL 8 usa:

sql

```
SELECT @@transaction_isolation;
```

MariaDB usa:

sql

```
SELECT @@tx_isolation;
```

Por eso cuando probaste:

sql

```
SELECT @@transaction_isolation;
```

te devolvió:

Error Code: 1193 – Unknown system variable 'transaction_isolation'

No es tu culpa.

Es simplemente que MariaDB tiene otra sintaxis.

● 2. Cómo ver el nivel de aislamiento en MariaDB

Tenés dos opciones válidas:

✓ Sesión actual:

sql

```
SELECT @@session.tx_isolation;
```

✓ Global:

sql

```
SELECT @@global.tx_isolation;
```

● 3. Cómo cambiar el nivel de aislamiento en MariaDB

También cambia la sintaxis respecto a MySQL.

✗ Esto NO funciona en MariaDB:

sql

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

✓ Esto SÍ funciona:

sql

```
SET SESSION tx_isolation = 'READ-COMMITTED';
```

● 4. Valores válidos en MariaDB

Son exactamente estos:

- 'READ-UNCOMMITTED'
- 'READ-COMMITTED'
- 'REPEATABLE-READ'
- 'SERIALIZABLE'

Y por defecto, MariaDB usa:

👉 **REPEATABLE-READ**

¿Qué diferencia hay entre READ COMMITTED y REPEATABLE READ?

Esta es **LA** pregunta clave de niveles de aislamiento.

Si entendés esta diferencia, entendés el corazón de la Etapa 5.

Te lo explico con claridad total y ejemplos simples.

★ Respuesta clara y directa

La diferencia principal es:

READ COMMITTED → cada SELECT ve **solo datos ya confirmados** (committed).

- Si otra transacción hace COMMIT, vos **sí** ves los cambios.

REPEATABLE READ → cada SELECT ve **siempre la misma foto** de los datos durante toda la transacción.

- Aunque otra transacción haga COMMIT, vos **no** ves los cambios hasta terminar.

★ Explicación guiada (como tutoría de IA)

● READ COMMITTED (lectura confirmada)

Es más flexible, más “realista”.

✓ Qué permite:

- Ver cambios confirmados por otras transacciones mientras la tuya sigue abierta.

✓ Qué evita:

- Lecturas sucias (dirty reads).

✗ Qué NO evita:

Lecturas no repetibles (non-repeatable reads).

- Es decir: **si lees dos veces, podés obtener valores distintos.**

🔍 Ejemplo:

Sesión A lee el stock → 10

Sesión B actualiza stock a 5 y hace COMMIT

Sesión A vuelve a leer → ahora ve 5

👉 **La lectura cambió.**

● REPEATABLE READ (lectura repetible)

Es más estricto, más seguro.

✓ Qué garantiza:

- Si lees un dato una vez, **lo verás igual durante toda la transacción.**
- Aunque otra transacción haga COMMIT, vos seguís viendo la “foto inicial”.

✓ Qué evita:

- Lecturas sucias
- Lecturas no repetibles

✗ Qué NO evita (en MySQL/MariaDB):

- Lecturas fantasma (aunque InnoDB usa next-key locks y suele evitarlas también)

🔍 Ejemplo:

Sesión A lee stock → 10

Sesión B actualiza stock a 5 y hace COMMIT

Sesión A vuelve a leer → **sigue viendo 10**

👉 **La lectura se mantiene estable.**

★ Resumen rápido

Nivel	¿Ve cambios de otras transacciones durante la transacción?	¿Evita lecturas no repetibles?
READ COMMITTED	Sí	 No
REPEATABLE READ	No	 Sí